

Hybrid Model-Driven Reinforcement Learning for Long-Term Task Scheduling in Satellite-Ground Edge Computing

Bocheng Yang, Lujie Zheng, Jian Liu, Bo Chen, *Member, IEEE*

Abstract—The growing volume of satellite data exposes the latency, bandwidth, and resource limitations of conventional ground-centric processing. Satellite-ground edge computing can alleviate these constraints, but its task-scheduling decisions are temporally coupled: an action with a low immediate cost may increase thermal load, deplete onboard energy, or occupy resources needed by subsequent tasks. We therefore develop a hybrid model-driven reinforcement-learning framework that separates immediate cost evaluation from long-term value learning. An explicit model quantifies thermal load, processing latency, bandwidth use, computational workload, and task co-location overhead for onboard, ground-based, and collaborative processing. A deep Q-network then estimates cumulative value under evolving energy, thermal, queue, and communication states. Epsilon reset, selective update, and replay-buffer flushing (RSF) are evaluated as optional training interventions. Across five independent model seeds and 20 held-out traces per seed, MD-DQN-RSF reduced cumulative cost by 17.28% relative to immediate-cost argmin (paired mean difference -68.35 ; 95% bootstrap CI, -77.85 to -60.38), principally by reducing queue-limit violations from 30.42 ± 2.58 to 1.60 ± 0.84 per episode. In a separate ten-seed confirmation after calibration-only configuration selection, full RSF reduced mean cost by 0.35% relative to standard model-driven DQN, but its paired 95% interval crossed zero. Reset alone reduced mean cost by 0.40%, with an interval excluding zero. The evidence therefore supports sequential value learning and identifies reset as the main promising intervention, while the incremental effects of selective update and flushing remain uncertain. Under zero action-dependent coupling, immediate argmin remained superior, defining the regime in which reinforcement learning is unnecessary.

Index Terms—Satellite edge computing; task scheduling; cost modeling; reinforcement learning.

I. INTRODUCTION

SATELLITE applications now span global communications, Earth observation, navigation, and remote sensing. Their continued expansion has increased both the volume of

data acquired in orbit and the demand for timely processing. Recent studies have therefore investigated onboard inference on resource-constrained platforms. Representative approaches include SOCNet [1], FFCA-YOLO [2], SOCDet [3], Pyramid Knowledge Distillation [4], and RepSViT [5]. Collectively, these methods show that model compression and architecture optimization can support accurate onboard remote-sensing inference with limited computational resources.

Nevertheless, many operational pipelines remain ground-centric and depend on manually configured schedules (Fig. 1). Satellites primarily collect data and downlink raw observations for processing in terrestrial data centers. This architecture can create communication bottlenecks, increase latency, underuse onboard resources, and limit autonomous in-orbit decision-making.

Satellite scheduling has been formulated using decentralized optimization [6], mixed-integer linear programming [7], [8], particle-swarm optimization [9], and reinforcement learning [10]. These methods address multi-satellite coordination, observation planning, or resource allocation under operational constraints. Energy-aware frameworks such as Phoenix [11], HMPS [12], and SECORS [13] further optimize deadlines, makespan, computing time, or battery use.

Most existing studies emphasize constellation-level coordination and optimize a limited set of time or energy metrics [6]–[8], [11]–[13]. However, single-satellite task allocation must also balance bandwidth use, onboard computation, thermal load, and co-location overhead. These quantities evolve over time and couple successive decisions. For example, onboard execution may reduce the latency of the current task while increasing thermal load and queue occupancy for later tasks. A unified model is therefore needed to compare processing modes, but a purely myopic minimum-cost rule cannot represent their delayed consequences.

To address this gap, we develop a hybrid model-driven reinforcement-learning framework for sequential scheduling between a satellite and ground infrastructure. The analytical component evaluates the immediate cost of each action using heat generation, floating-point operations (FLOPs), processing and transmission latency, co-location overhead, and bandwidth use. The reinforcement-learning component estimates the long-term value of an action after accounting for its effect on future resource availability. Table I summarizes the rationale for the immediate cost factors.

Latency constrains time-sensitive applications [14], whereas thermal load affects hardware safety in space [15]. Limited downlink bandwidth can bottleneck data-intensive tasks [16]. Co-located services also compete for shared resources and can reduce overall utility [17]. Finally, onboard computation draws

This work was supported by the National Key Research and Development Program of China under Grants 2022YFF0503904 and 2022YFD2401202; the Shenzhen Higher Education Institutions Stabilization Support Program under Grant GXWD20220811163556003; the National Natural Science Foundation of China under Grant 62202127; the Shenzhen Natural Science Foundation under Grant JCYJ20241202123731040; and the Guangxi Science and Technology Base and Talent Special Project, “Research and Application of Key Technologies for Precise Navigation,” under Grant Gui Ke AD25069103. (Corresponding authors: Bo Chen and Jian Liu.)

B. Yang is with the School of Aerospace, Harbin Institute of Technology (Shenzhen), Shenzhen 518055, China (e-mail: 25b961001@stu.hit.edu.cn).

L. Zheng is with the School of Aerospace, Harbin Institute of Technology (Shenzhen), Shenzhen 518055, China (e-mail: 24s061021@stu.hit.edu.cn).

J. Liu is with the School of Aerospace, Harbin Institute of Technology (Shenzhen), Shenzhen 518055, China, and the Guangxi Key Laboratory of Precision Navigation Technology and Application, Guilin University of Electronic Technology, Guilin 541004, China (e-mail: liujian2024@hit.edu.cn).

B. Chen is with the School of Aerospace, Harbin Institute of Technology (Shenzhen), Shenzhen 518055, China (e-mail: hitchenbo@hit.edu.cn).

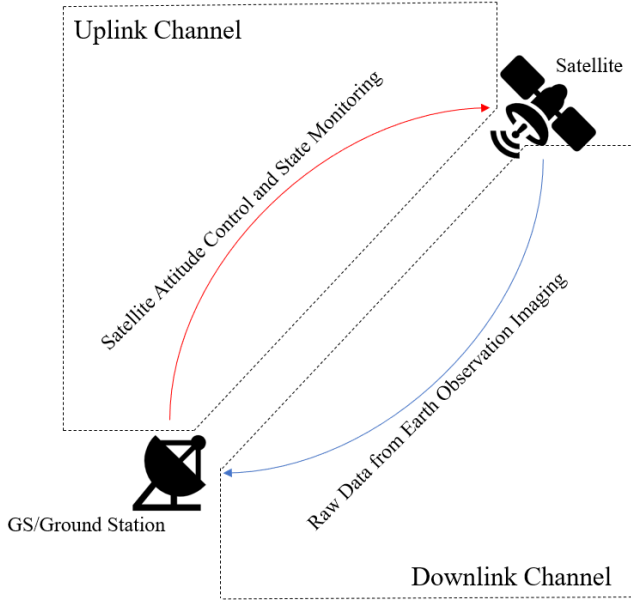


Fig. 1. The traditional manually scheduled satellite computing scheme.

TABLE I
KEY COST FACTORS IN SATELLITE COMPUTATIONAL TASK MODELING

Cost Factor	Rationale
Thermal Output	Strict thermal management is essential in space environments to maintain satellite functionality and prevent system failures.
Computational Load	Limited onboard computing capacity necessitates offloading computationally intensive tasks to ground stations.
Processing Latency	Time-sensitive remote sensing applications require strict adherence to real-time processing constraints.
Co-location Overhead	Concurrent execution of multiple tasks on a single satellite leads to resource contention and reduced efficiency.
Bandwidth Usage	Precious uplink/downlink capacity between satellite and ground requires careful allocation and conservation.

from a finite energy budget [17]. These constraints motivate their joint treatment in the proposed model.

The main contributions of our work are summarized as follows:

- 1) We establish an interpretable immediate-cost model that jointly quantifies thermal load, computation, latency, co-location overhead, and bandwidth use across three processing modes.
- 2) We formulate satellite-ground task allocation as a sequential decision process. The state combines task descriptors with evolving thermal, energy, queue, computing, and communication resources, allowing the policy to represent delayed action consequences.
- 3) We implement MD-DQN-RSF, which uses model-derived immediate rewards and learns long-term action

values. Epsilon reset, selective parameter updates, and replay-buffer flushing are evaluated through cumulative ablation rather than assumed to improve the base learner.

II. NOMENCLATURE

The subscripts s , g , and $s\&g$ denote onboard execution, ground-based execution, and satellite-ground collaborative processing, respectively.

TABLE II
NOTATION DEFINITION

Scheduling Strategy	State Symbol	State Description
Task processed on the satellite	Q_s	Heat cost
	$FLOPs_s$	Computational cost (floating-point operations)
	t_s^{proc}	Task processing latency
	t_s^{trans}	Result transmission latency
	$f(n)_s$	Co-location cost (when multiple tasks run simultaneously)
Task processed on the ground	B_s	Bandwidth occupation cost
	Q_g	Heat cost
	t_g	Data transmission latency
	B_g	Bandwidth occupation cost
Task preprocessed onboard and further processed on the ground	$Q_{s\&g}^{pre}$	Heat cost during preprocessing
	$Q_{s\&g}^{trans}$	Heat cost during transmission
	$t_{s\&g}^{pre}$	Preprocessing latency
	$t_{s\&g}^{trans}$	Data transmission latency
	$FLOPs_{s\&g}$	Computational cost (floating-point operations)
	$B_{s\&g}$	Bandwidth occupation cost

Table II lists the task-dependent inputs used by the immediate-cost model. The sequential scheduler additionally uses the dynamic resource variables in Table III.

TABLE III
DYNAMIC RESOURCE-STATE VARIABLES

Symbol	Definition
h_t	Normalized onboard thermal load at decision step t
e_t	Normalized remaining onboard energy
q_t	Task-queue occupancy
u_t	Onboard computing utilization
b_t	Available satellite-ground downlink bandwidth
τ_t	Remaining communication contact-window duration
A_t	Workload arriving between decision steps t and $t + 1$
μ_t	Workload completed during the current transition

III. MODEL-BASED IMMEDIATE COST QUANTIFICATION

This section formulates an analytical model for the immediate operational cost of a scheduling action. It assigns dimensionless coefficients and relative weights to five factors, enabling quantities with different units to be aggregated within a common objective. The model does not determine the final policy by itself. Instead, it supplies an interpretable one-step cost to the reinforcement-learning framework developed in Section IV.

A. Cost Factor Modeling

1) *Thermal Cost Factor Analysis*: The thermal cost is defined as

$$C_1 = \begin{cases} \rho_{heat} \cdot \omega_{heat} \cdot \tan\left(\frac{\pi}{2} \cdot \frac{Q}{Q_{MAX}}\right), & Q < Q_{MAX} \\ +\infty, & Q \geq Q_{MAX} \end{cases} \quad (1)$$

Here, C_1 is the thermal component of the total cost. The dimensionless coefficient ρ_{heat} normalizes this component against quantities with different units. The weight ω_{heat} controls its relative importance. Moreover, Q is task-related heat generation in joules, and Q_{MAX} is the allowable thermal threshold.

The tangent penalty increases nonlinearly as Q approaches Q_{MAX} and becomes infinite at the threshold. This form prevents the scheduler from selecting actions that violate the specified thermal constraint.

2) *Computational Load Cost Factor Analysis*: The computational-load cost is defined as

$$C_2 = \rho_{comp} \cdot \omega_{comp} \cdot \text{FLOPs} \quad (2)$$

Here, C_2 denotes computational workload, and ρ_{comp} and ω_{comp} are its dimensionless coefficient and weight. FLOPs denotes the number of floating-point operations required by a task, measured in mega floating-point operations (MFLOPs).

3) *Processing Latency Cost Factor Analysis*: The latency component combines processing and transmission delays:

$$C_3 = \rho_{time} \cdot (\omega_{proc} \cdot t^{proc} + \omega_{trans} \cdot t^{trans}) \quad (3)$$

Here, C_3 is the latency cost, and ρ_{time} is its dimensionless coefficient. The weights ω_{proc} and ω_{trans} correspond to processing and transmission latency. The variables t^{proc} and t^{trans} measure these delays in milliseconds.

4) *Co-location Overhead Cost Factor Analysis*: The co-location overhead is defined as

$$C_4 = \rho_{coloc} \cdot \omega_{coloc} \cdot f(n) \quad (4)$$

Here, C_4 is the co-location cost, with dimensionless coefficient ρ_{coloc} and weight ω_{coloc} . The function $f(n)$ describes how overhead increases with the number n of tasks executed concurrently onboard.

5) *Bandwidth Usage Cost Factor Analysis*: The bandwidth cost is defined as

$$C_5 = \begin{cases} \rho_{bw} \cdot \omega_{bw} \cdot \tan\left(\frac{\pi}{2} \cdot \frac{B}{B_{MAX}}\right), & B < B_{MAX} \\ +\infty, & B \geq B_{MAX} \end{cases} \quad (5)$$

Here, C_5 is the bandwidth-occupancy cost, with dimensionless coefficient ρ_{bw} and weight ω_{bw} . The variable B is the required downlink bandwidth in megabits per second, and B_{MAX} is the available downlink capacity.

As with the thermal term, the tangent penalty increases nonlinearly and becomes infinite when $B = B_{MAX}$. This form discourages actions that approach or exceed the available downlink capacity.

B. Immediate Cost Modeling in the Action Space

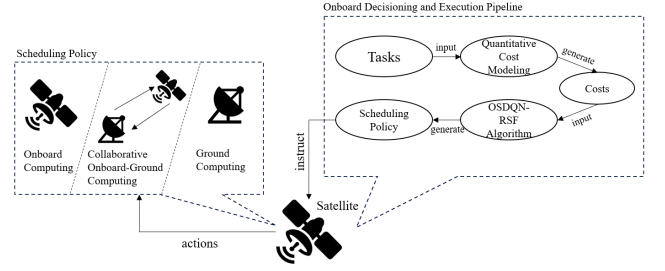


Fig. 2. A schematic diagram of satellite in-orbit decision-making based on model-driven actions within the action space.

The action space $\mathcal{A}$ contains three processing modes (Fig. 2):

$$\mathcal{A} = \{\alpha_s, \alpha_g, \alpha_{s\&g}\} \quad (6)$$

- 1) α_s : the task is processed onboard, and only its result is downlinked.
- 2) α_g : all required data are downlinked and processed on the ground.
- 3) $\alpha_{s\&g}$: the data are preprocessed onboard, and the resulting subset is downlinked for further ground-based processing.

The following subsections define an immediate cost for each action. At decision step t , the analytical model receives the current task descriptor and resource state, denoted by x_t and z_t , respectively. It then evaluates

$$c_t^{\text{imm}} = c_{\text{model}}(x_t, z_t, a_t), \quad (7)$$

where $a_t \in \mathcal{A}$. The action-specific expressions below define c_{model} . Their input variables are recomputed at each decision step from the current task and available resources. Section IV augments this immediate cost with the future consequences of the selected action.

The analytical inputs are therefore conditional estimates rather than fixed task labels. Specifically, the effective transmission latency and bandwidth terms depend on b_t and τ_t , while the co-location term depends on q_t and u_t . The action-related heat $Q^{(a_t)}$ and workload determine the increments $\Delta h(x_t, a_t)$ and $\Delta e(x_t, a_t)$ used by the transition model. In the dynamic formulation, B_{MAX} is replaced by the downlink capacity available at step t . This coupling ensures that the same arriving task can have different immediate costs under different resource states.

1) *Cost Modeling on the Satellite Side*: For onboard execution, the total cost comprises

- heat generation cost during task processing,
- computational workload cost measured in floating-point operations,
- task processing time cost on the satellite,
- data transmission delay cost for returning the processed results to the ground,
- co-location cost when multiple tasks are processed simultaneously on the same satellite,

- bandwidth occupancy cost resulting from transmitting the task results to the ground.

The total onboard-execution cost is therefore

$$\begin{aligned} \text{Cost}_s = & \rho_{\text{heat}}\omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q_s}{Q_{\text{MAX}}}\right) + \rho_{\text{comp}}\omega_{\text{comp}} \text{FLOPs}_s \\ & + \rho_{\text{time}} (\omega_{\text{proc}}t_s^{\text{proc}} + \omega_{\text{trans}}t_s^{\text{trans}}) + \rho_{\text{coloc}}\omega_{\text{coloc}} f(n)_s \\ & + \rho_{\text{bw}}\omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B_s}{B_{\text{MAX}}}\right) \end{aligned} \quad (8)$$

Here, Cost_s denotes the total cost of onboard execution. The remaining variables are defined in Section II and in the preceding cost-factor formulations.

2) *Ground-Side Cost Modeling*: For ground-based execution, the modeled costs comprise

- the thermal cost associated with downlink data transmission from the satellite to the ground,
- the latency cost incurred during transmission,
- the bandwidth occupation cost.
- the computational consumption, heat generation, and latency during computational processes on ground station servers.

The ground-based execution cost is therefore

$$\begin{aligned} \text{Cost}_g = & \rho_{\text{heat}}\omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q_g}{Q_{\text{MAX}}}\right) \\ & + \rho_{\text{time}}\omega_{\text{trans}} t_g \\ & + \rho_{\text{bw}}\omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B_g}{B_{\text{MAX}}}\right) \\ & + \text{cost}_{\text{ground_server}} \end{aligned} \quad (9)$$

The term $\text{cost}_{\text{ground_server}}$ represents computation, heat generation, and processing latency at the ground station.

3) *Satellite-Ground Collaborative Cost Modeling*: For collaborative processing, a task is first preprocessed onboard and then transferred to the ground. The total cost comprises

- heat generation cost during onboard preprocessing,
- time cost for preprocessing onboard,
- computational workload cost (FLOPs) for onboard preprocessing,
- transmission delay for sending the preprocessed data to the ground,
- heat generation cost associated with data transmission from the satellite to the ground,
- bandwidth occupancy cost for downlink transmission.
- the computational consumption, heat generation, and latency during computational processes on ground station servers.

The collaborative-processing cost is therefore

$$\begin{aligned} \text{Cost}_{s\&g} = & \rho_{\text{heat}}\omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q_{s\&g}^{\text{pre}} + Q_{s\&g}^{\text{trans}}}{Q_{\text{MAX}}}\right) \\ & + \rho_{\text{time}} (\omega_{\text{pre}}t_{s\&g}^{\text{pre}} + \omega_{\text{trans}}t_{s\&g}^{\text{trans}}) \\ & + \rho_{\text{comp}}\omega_{\text{comp}} \text{FLOPs}_{s\&g} \\ & + \rho_{\text{bw}}\omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B_{s\&g}}{B_{\text{MAX}}}\right) \\ & + \text{cost}_{\text{ground_server}} \end{aligned} \quad (10)$$

The term $\text{cost}_{\text{ground_server}}$ has the same definition as in Equation (9).

Together, these functions permit direct comparison of the immediate costs of onboard, ground-based, and collaborative processing. Selecting the minimum immediate cost provides a transparent myopic baseline, but it does not account for how the selected action changes the costs of subsequent tasks.

IV. HYBRID MODEL-DRIVEN REINFORCEMENT LEARNING

The analytical model in Section III evaluates the immediate cost of each processing mode. A scheduler that minimizes this cost independently for every task is myopic because the selected action changes the resources available to subsequent tasks. We therefore formulate scheduling as a Markov decision process and use the analytical model inside the reward. The resulting MD-DQN-RSF framework preserves an interpretable cost decomposition while learning the long-term opportunity cost of each action.

A. Sequential Decision Formulation

1) *State Space*: At decision step t , the state is

$$s_t = [x_t, z_t], \quad (11)$$

where x_t contains the task-dependent variables defined in Section II. The dynamic resource state is

$$z_t = [h_t, e_t, q_t, u_t, b_t, \tau_t]. \quad (12)$$

Here, h_t is the normalized onboard thermal load, e_t is the normalized remaining energy, q_t is the task-queue occupancy, and u_t is onboard computing utilization. The variables b_t and τ_t denote available downlink bandwidth and remaining contact-window duration. This separation distinguishes the properties of the arriving task from resources inherited from previous decisions.

2) *Action Space*: The action space remains $\mathcal{A} = \{\alpha_s, \alpha_g, \alpha_{s\&g}\}$. These actions correspond to onboard execution, ground-based execution, and onboard preprocessing followed by ground-based processing, respectively.

3) *State Transition*: Executing a_t changes the resource state before the next task arrives:

$$z_{t+1} = F(z_t, x_t, a_t, \xi_t), \quad (13)$$

where F is the satellite-resource transition model and ξ_t represents exogenous task arrivals and link variation. To make the sequential experiments exactly reproducible, all variables are normalized and the implemented transition is

$$\begin{aligned} h_{t+1} &= \text{clip}\left(0.86h_t + 0.17c\widehat{Q}_t^{(a_t)}, 0, 1.3\right), \\ e_{t+1} &= \text{clip}\left(e_t + 0.006 - c\Delta e_t^{(a_t)}, 0, 1\right), \\ q_{t+1} &= \text{clip}\left(q_t + c(\Delta q_t^{(a_t)} - \mu_t^{(a_t)}), 0, 1.3\right), \\ u_{t+1} &= \text{clip}\left(0.68u_t + 0.32c\widehat{w}_t^{(a_t)}, 0, 1.2\right), \\ b_{t+1} &= \text{clip}\left(\bar{b}_{t+1} - c\ell_t^{(a_t)}, 0.08, 1\right), \\ \tau_{t+1} &= \text{clip}\left(\bar{\tau}_{t+1} - 0.10c\widehat{B}_t^{(a_t)}, 0.05, 1\right). \end{aligned} \quad (14)$$

Here, $\widehat{Q}_t^{(a)}$, $\widehat{w}_t^{(a)}$, and $\widehat{B}_t^{(a)}$ are the action-specific heat, workload, and data-demand terms normalized by their sampling maxima. The functions $\Delta e_t^{(a)}$, $\Delta q_t^{(a)}$, $\mu_t^{(a)}$, and $\ell_t^{(a)}$ are deterministic functions of these normalized demands and the current utilization and link state; their implementation is released with the experiment code. The exogenous link variables $\bar{b}_t$ and $\bar{\tau}_t$ follow noisy periodic traces generated before an episode begins. The coefficient c controls temporal coupling: $c = 0$ removes all action-dependent state transitions and provides the static control, whereas $c = 1$ is the nominal sequential setting. Values above one represent stronger-than-nominal coupling.

4) *Model-Driven Reward*: The analytical model provides the immediate cost c_t^{imm} in Equation (7). To represent operational feasibility, the reward adds continuous hinge penalties for thermal, energy, queue, and contact-window violations:

$$\begin{aligned} r_t = & -c_t^{\text{imm}} - c[12(h_{t+1} - 0.78)_+ + 15(0.18 - e_{t+1})_+ \\ & + 8(q_{t+1} - 0.75)_+ + 10\mathbf{1}_{a_t \neq \alpha_s}(0.14 - \tau_t)_+], \end{aligned} \quad (15)$$

where $(v)_+ = \max(v, 0)$. Unlike a direct minimum-cost rule, the action-value function includes future resource consequences:

$$Q^\pi(s_t, a_t) = \mathbb{E}_\pi \left[\sum_{k=0}^{H-t} \gamma^k r_{t+k} \mid s_t, a_t \right]. \quad (16)$$

The distinction between c_t^{imm} and Q^π is central to the hybrid design. The former remains physically interpretable, whereas the latter represents the expected long-term effect of an action under stochastic task and communication conditions.

B. Model-Driven DQN Architecture and RSF Interventions

The action-value network receives the 21-dimensional combined task-resource state and outputs one value for each processing mode. Task inputs are divided by their predefined sampling maxima, and resource inputs are already normalized.

The implemented multilayer perceptron has hidden widths of 128, 128, and 64 with rectified-linear activations, followed by three linear outputs. A target network is synchronized every 250 environment steps. Figure 3 shows how the analytical immediate-cost model and learned long-term value are combined.

MD-DQN-RSF extends the standard DQN [18]–[20] with epsilon reset, selective parameter updates, and replay-buffer flushing. Their contribution is evaluated individually rather than inferred from the full model alone.

1) *Motivation for the RSF Mechanisms*: The standard DQN can exhibit inefficient exploration and unstable learning when task arrivals and link conditions produce uneven state coverage. Epsilon reset reintroduces exploration after the network has learned an initial policy. Selective updates limit the influence of high-loss exploratory batches during the reset phase. Replay-buffer flushing subsequently removes older exploratory transitions. These mechanisms are hypotheses rather than assumed improvements; Section VI reports their cumulative ablation.

2) *Epsilon Reset*: The baseline DQN uses an epsilon-greedy policy. Epsilon decays linearly from 1.0 to 0.05 over the first 35% of the total training steps. In the nominal ablation, the reset variants restore ϵ to 0.65 at 45% of the training episodes and decay it to 0.05 a second time. The independent confirmation study calibrates this schedule separately, as described in Section V-F. The reset introduces new exploratory transitions after an initial policy has formed.

3) *Selective Update*: In the baseline DQN, every scheduled mini-batch produces a backpropagation update. After epsilon reset, MD-DQN-RSF accepts an update only when its Huber loss does not exceed κ times the exponential moving average of previously accepted losses. The moving average uses a coefficient of 0.98. The nominal ablation sets $\kappa = 1.8$, whereas the confirmation study uses its calibration-locked value. This adaptive rule avoids a scale-dependent fixed loss threshold.

4) *Flushing Replay Buffer*: The replay buffer stores state, action, reward, next-state, and terminal-indicator tuples. In the nominal ablation, the full method clears the buffer at 72% of the training episodes and repopulates it under the post-reset policy. Whether this loss of older coverage is offset by faster adaptation is tested empirically in both the nominal and independent ablations.

Algorithm 1 summarizes the complete procedure. The analytical model enters the algorithm only through the immediate cost and constraint terms. The neural network is therefore not used to approximate a minimum over three known static costs; it estimates how each action changes expected cumulative performance through the transition dynamics.

V. EXPERIMENTAL DESIGN AND EVALUATION PROTOCOL

The experiments are designed to test whether learning long-term action values provides an advantage over minimizing the immediate analytical cost. The evaluation therefore separates four questions: whether MD-DQN-RSF improves long-term performance, whether the improvement arises from sequential resource coupling, which RSF mechanisms contribute to learning, and where the learned policy fails.

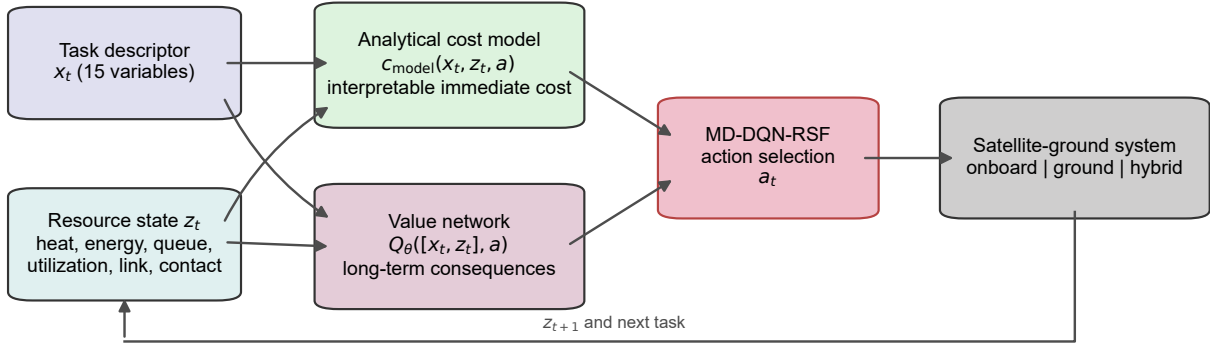


Fig. 3. Hybrid model-driven scheduling architecture. The analytical branch evaluates the interpretable immediate cost of all three processing modes, whereas the value network estimates their cumulative consequences through the dynamic resource state. The selected action updates the satellite-ground system and hence the state presented with the next arriving task.

Algorithm 1: MD-DQN-RSF

```

1 Initialize replay memory  $\mathcal{D}$  to capacity  $N$ ;
2 Initialize online network  $Q_\theta$  and target network
    $Q_{\bar{\theta}} \leftarrow Q_\theta$ ;
3 for  $episode = 1$  to  $M$  do
4   Apply the scheduled epsilon reset or replay-buffer
     flush, if due;
5   Initialize the dynamic environment and observe
      $s_1 = [x_1, z_1]$ ;
6   for  $t = 1$  to  $T$  do
7     With probability  $\epsilon$ , select a random action  $a_t$ ;
8     Otherwise select  $a_t = \arg \max_a Q_\theta(s_t, a)$ ;
9     Evaluate  $c_t^{\text{imm}}$  with the analytical model;
10    Execute  $a_t$  and observe  $r_t$ ,  $s_{t+1}$ , and terminal
       indicator  $d_t$ ;
11    Store transition  $(s_t, a_t, r_t, s_{t+1}, d_t)$  in  $\mathcal{D}$ ;
12    if  $t$  is an update step and  $|\mathcal{D}| \geq \text{batch size}$  then
13      Sample a random mini-batch
         $(s_j, a_j, r_j, s_{j+1}, d_j)$  from  $\mathcal{D}$ ;
14      Set  $y_j = r_j + \gamma(1 - d_j) \max_{a'} Q_{\bar{\theta}}(s_{j+1}, a')$ ;
15      Compute the Huber loss
         $L = \mathcal{L}_{\text{Huber}}(y_j, Q_\theta(s_j, a_j))$ ;
16      if selective update is inactive OR  $L < \kappa \bar{L}$ 
        then
17        Update  $\theta$  by one Adam step on  $L$ ;
18      end
19    end
20  end
21  Copy  $\theta$  to  $\bar{\theta}$  every  $C$  environment steps;
22 end

```

A. Dynamic Scheduling Scenarios

Each episode contains 64 arriving remote-sensing tasks. The 15 task variables follow the ranges in Table IV, which preserve the ranges used by the original one-step simulator. All task and exogenous link traces are generated at reset; consequently, policies evaluated with the same seed receive identical arrivals even though their endogenous resource states diverge. Training episode k for model seed m uses seed $10^6 m + k$. The 100,000-series seeds were used only for calibration, whereas final testing uses 20 disjoint 500,000-series traces for each model seed.

For numerical evaluation, the coefficient-weight products in the immediate model are 0.4 for onboard and collaborative tangent heat and bandwidth terms, 0.0004 for FLOPs, 0.0002 for onboard processing and result-transfer latency, and 0.004 for co-location overhead. Ground execution uses 2/3 for the tangent heat term, 0.0005 for transfer latency, and 0.4 for the tangent bandwidth term. Collaborative preprocessing and transfer latencies both use 0.0004. The effective onboard processing latency is multiplied by $(1 + 1.4u_t + 0.7q_t)$, collaborative preprocessing latency by $(1 + 0.8u_t)$, co-location overhead by $(1 + q_t)$, and ground/collaborative transfer latency by $1/\max(b_t, 0.08)$. State surcharges of $0.35h_t + 0.12(1 - e_t)$ for onboard execution and $0.18h_t + 0.12q_t$ for collaborative execution are included. Link-dependent actions also incur hinge surcharges when τ_t is below 0.28 (ground) or 0.22 (collaborative). Ground-server computation cost is set to zero in this simulator; conclusions therefore concern satellite-side and link resource allocation rather than terrestrial energy optimization.

In the nominal scenario, available bandwidth follows $\bar{b}_t = 0.62 + 0.23 \sin(2\pi t/24 + \phi) + \varepsilon_t$, where ϕ is uniform on $[0, 2\pi)$, $\varepsilon_t \sim \mathcal{N}(0, 0.06^2)$, and the result is clipped to $[0.12, 1]$. Contact duration is generated analogously with mean 0.55, amplitude 0.35, period 31, Gaussian noise with standard deviation 0.04, and clipping to $[0.08, 1]$. Initial $(h, e, q, u) = (0.22, 0.88, 0.12, 0.18)$. The burst condition multiplies heat- and compute-related demands by 1.45 during one contiguous

TABLE IV
TASK-VARIABLE RANGES IN EACH 64-STEP EPISODE. ALL CONTINUOUS VARIABLES ARE SAMPLED UNIFORMLY BEFORE SCENARIO-SPECIFIC PERTURBATIONS.

Mode	Heat and computation	Processing/transfer latency	Co-location or bandwidth
Onboard	$Q_s, \text{FLOPs}_s \in [0, 1000]$	$t_s^{proc} \in [0, 1000], t_s^{trans} \in [200, 300]$	$n \in \{0, \dots, 49\}, f(n) = n^2, B_s \in [0, 1000]$
Ground	$Q_g \in [0, 1000]$	$t_g \in [200, 20000]$	$B_g \in [0, 1000]$
Collaborative	$Q_{s\&g}^{pre} + Q_{s\&g}^{trans} \in [0, 1000], \text{FLOPs}_{s\&g} \in [0, 1000]$	$t_{s\&g}^{pre} \in [0, 1000], t_{s\&g}^{trans} \in [200, 20000]$	$B_{s\&g} \in [0, 1000]$

quarter of the episode. The link-limited condition subtracts 0.22 from available bandwidth before clipping. Energy-limited and thermal-stress episodes begin with $e_0 = 0.62$ and $h_0 = 0.42$, respectively; thermal stress also includes the burst perturbation.

B. Comparison Methods

MD-DQN-RSF is compared under identical task traces and cost parameters with the following policies:

- 1) *Immediate-cost argmin*, which evaluates the three analytical costs and selects the lowest-cost action at each step. This baseline isolates the value of long-term learning from the value of the cost model itself.
- 2) *Fixed onboard* and *fixed ground*, which assign every task to one processing location and provide operational reference points.
- 3) *Threshold heuristic*, which avoids onboard execution when $h_t > 0.72$ or $e_t < 0.24$, avoids link-dependent actions when $b_t < 0.22$ or $\tau_t < 0.14$, selects collaborative execution when $q_t > 0.70$, and otherwise uses the immediate argmin.
- 4) *Contextual bandit*, which receives the same state variables but optimizes only the immediate reward without temporal bootstrapping.
- 5) *Standard DQN*, which uses the same network, state, reward, and training budget without the RSF mechanisms.
- 6) *MD-DQN-RSF*, the complete proposed method.

C. Evaluation Metrics

The primary endpoint is the undiscounted cumulative operational cost per episode. Secondary endpoints are the immediate-cost and penalty components, the numbers of thermal, energy, queue, and contact-window violations, and the fraction of tasks assigned to each processing mode. These metrics distinguish a policy that merely minimizes the current analytical cost from one that preserves resources for future tasks.

D. Statistical Analysis

Five independently initialized model seeds constitute the experimental units. Within each seed, every policy is evaluated on the same 20 held-out traces; trace metrics are first averaged within the seed and are not treated as independent replicates. Results are summarized as the mean and standard deviation across the five seed-level values. The primary MD-DQN-RSF versus immediate-argmin effect is a paired seed-level mean difference with a two-sided 95% percentile

TABLE V
TRAINING HYPERPARAMETERS FOR THE NOMINAL EXPERIMENT

Parameter	Value
Episodes; steps per episode	600; 64
Optimizer; learning rate	Adam; 3×10^{-4}
Discount factor γ	0.97
Replay capacity; batch size	100,000; 128
Gradient/target update interval	4/250 steps
Initial/minimum ϵ	1.0/0.05
Reset episode; reset ϵ	270; 0.65
Flush episode	432
Selective threshold	$1.8\bar{L}$

bootstrap confidence interval from 10,000 resamples (fixed bootstrap seed 20260819). No null-hypothesis significance test or multiplicity-adjusted claim is made for the exploratory secondary comparisons.

Hyperparameters were fixed using the calibration traces before the 500,000-series test traces were generated. Each model was then trained for the prespecified 600 episodes; the terminal model, rather than the best test episode, was evaluated. Experiments used Python 3.13.9, PyTorch 2.8.0+cu129, and one NVIDIA RTX 5070 Ti GPU in the `yolo` Conda environment. Table V reports the common training configuration.

E. Ablation and Sensitivity Analysis

The cumulative ablation begins with standard model-driven DQN and adds epsilon reset (R), selective updates (R+S), and replay-buffer flushing (R+S+F). Robustness analyses vary temporal coupling from 0 to 1 and evaluate burst, link-limited, energy-limited, and thermal-stress scenarios. A mechanism is considered beneficial only when its improvement is consistent across independent seeds and is not obtained by increasing constraint violations.

F. Independent RSF Confirmation Protocol

We conducted a separate two-stage study to distinguish configuration selection from final RSF assessment. The calibration stage compared six prespecified RSF schedules using four model seeds (100–103) and 20 traces per seed from the 700,000-series namespace. The selection endpoint was the mean seed-level paired total-cost difference between full MD-DQN-RSF and standard MD-DQN. Calibration selected the `c4_early_moderate` schedule, which reset epsilon to 0.50 at 35% of training, set $\kappa = 2.5$, and flushed the replay buffer at 85% of training.

The selected schedule was locked before the confirmation stage. Standard MD-DQN, DQN-R, DQN-RS, and full MD-

TABLE VI
NOMINAL TEST PERFORMANCE. VALUES ARE MEAN $\pm$ SD ACROSS FIVE INDEPENDENT MODEL SEEDS AFTER AVERAGING 20 PAIRED TRACES WITHIN EACH SEED.

Policy	Total cost	Immediate cost	Penalty	Queue violations
Immediate argmin	395.55 $\pm$ 16.24	230.05 $\pm$ 3.35	165.51 $\pm$ 14.17	30.42 $\pm$ 2.58
Contextual bandit	388.97 $\pm$ 15.17	229.58 $\pm$ 3.54	159.39 $\pm$ 13.03	29.60 $\pm$ 2.66
Standard MD-DQN	326.45 $\pm$ 6.62	250.58 $\pm$ 5.39	75.88 $\pm$ 2.64	0.98 $\pm$ 0.90
MD-DQN-RSF	327.20 $\pm$ 6.52	250.28 $\pm$ 4.93	76.93 $\pm$ 3.08	1.60 $\pm$ 0.84

DQN-RSF were then trained with ten new model seeds (200–209). Each model was evaluated on 20 disjoint traces per seed from the 900,000-series namespace. Trace values were averaged within each model seed, and all ten seeds were retained as the independent units. Paired mean differences relative to standard MD-DQN used two-sided 95% percentile bootstrap intervals from 10,000 resamples with fixed seed 20260820. No confirmatory seed was selected, excluded, or replaced according to its observed performance.

G. Evidence Required to Establish Reinforcement-Learning Value

The central comparison is between MD-DQN-RSF and immediate-cost argmin. Under independent static tasks, both methods are expected to select similar actions, and no reinforcement-learning advantage is claimed. The proposed approach is supported only if it reduces cumulative cost or constraint violations when decisions are temporally coupled. The analysis will therefore report performance as the strength of resource coupling increases from static tasks to burst arrivals, constrained links, and stressed onboard resources.

VI. RESULTS

A. Long-Term Cost and Constraint Violations

Table VI reports the prespecified nominal comparison. MD-DQN-RSF reduced cumulative cost from 395.55 $\pm$ 16.24 for immediate-cost argmin to 327.20 $\pm$ 6.52, a 17.28% reduction. The paired seed-level mean difference was -68.35 (95% bootstrap CI, -77.85 to -60.38). This improvement was not obtained by finding a lower immediate analytical cost: MD-DQN-RSF accepted an 8.80% higher immediate component (250.28 versus 230.05) but reduced the delayed penalty component by 53.52% (76.93 versus 165.51).

The mechanism is visible in Fig. 4b. MD-DQN-RSF reduced queue-limit violations from 30.42 $\pm$ 2.58 to 1.60 $\pm$ 0.84 per episode and energy-limit violations from 38.37 $\pm$ 0.57 to 32.07 $\pm$ 0.90. Thermal and contact violations were already infrequent. The contextual bandit had nearly the same immediate cost as immediate argmin but retained 29.60 $\pm$ 2.66 queue violations, indicating that one-step reward optimization did not capture the delayed queue consequence. Fixed onboard, fixed ground, and threshold policies incurred cumulative costs of 653.03 $\pm$ 12.27, 1018.37 $\pm$ 23.25, and 708.65 $\pm$ 21.77, respectively.

B. When Reinforcement Learning Is Necessary

The coupling control provides the critical boundary test (Fig. 4c). When action-dependent transitions and delayed

TABLE VII
INDEPENDENT RSF CONFIRMATION AFTER CALIBRATION-ONLY CONFIGURATION SELECTION. TOTAL COST IS MEAN $\pm$ SD ACROSS TEN MODEL SEEDS AFTER AVERAGING 20 TRACES WITHIN EACH SEED. DIFFERENCES ARE PAIRED AGAINST STANDARD MD-DQN; NEGATIVE VALUES FAVOR THE ABLATION VARIANT.

Policy	Total cost	Paired difference	95% bootstrap CI
Standard MD-DQN	321.65 $\pm$ 7.22	Reference	—
DQN-R	320.36 $\pm$ 6.53	-1.29	$[-2.44, -0.25]$
DQN-RS	320.44 $\pm$ 6.54	-1.21	$[-2.92, 0.24]$
MD-DQN-RSF	320.52 $\pm$ 6.73	-1.13	$[-3.09, 1.00]$

penalties were removed ($c = 0$), immediate argmin achieved 177.77 $\pm$ 2.60, whereas the policy trained at $c = 1$ achieved 211.07 $\pm$ 6.00. At $c = 0.5$, immediate argmin also remained better (199.93 $\pm$ 2.50 versus 224.61 $\pm$ 5.19). The ordering reversed only in the fully coupled nominal setting, where delayed penalties were sufficiently large for value learning to offset its higher immediate cost. Thus, these experiments do not support reinforcement learning for independent per-task optimization; they support it specifically when current actions materially alter later resource feasibility. Because the same $c = 1$ agents were transferred to the lower-coupling controls without retraining, the $c = 0$ and $c = 0.5$ values quantify policy-transfer degradation rather than the best attainable DQN at those coupling levels.

The advantage persisted under all four out-of-distribution stress tests. Relative to immediate argmin, MD-DQN-RSF reduced cumulative cost by 22.22% under burst arrivals, 15.20% under limited links, 14.67% with reduced initial energy, and 22.20% under thermal stress. Absolute cost remained highest under the link-limited condition (447.71 $\pm$ 11.59), identifying communication scarcity as the most difficult tested regime.

C. RSF Ablation and Learning Dynamics

In the nominal five-seed ablation, standard MD-DQN achieved 326.45 $\pm$ 6.62. Adding reset, selective update, and flushing produced 326.63 $\pm$ 5.92, 326.59 $\pm$ 5.68, and 327.20 $\pm$ 6.52, respectively (Fig. 4d). The full variant was 0.23% worse than standard MD-DQN. Figure 5 further shows that epsilon reset temporarily increased exploratory cost, after which both learners converged to similar levels.

The independent confirmation produced a different mean ordering after calibration-only schedule selection (Table VII). Full MD-DQN-RSF reduced total cost by 1.13 units, or 0.35%, relative to standard MD-DQN. However, its paired 95% interval (-3.09 to 1.00) included zero. DQN-RS also had an interval that included zero. Reset alone reduced cost by 1.29 units, or 0.40%, with a paired interval of -2.44 to -0.25 . Thus, reset showed the most consistent benefit in this study, but adding selective update and flushing did not yield a monotonic incremental improvement.

Taken together, the two ablations do not establish a stable advantage for the complete RSF package over standard MD-DQN. The defensible contribution remains the hybrid sequential formulation. Among the optional interventions, reset warrants further study, whereas the incremental value of selective update and replay-buffer flushing remains unresolved.

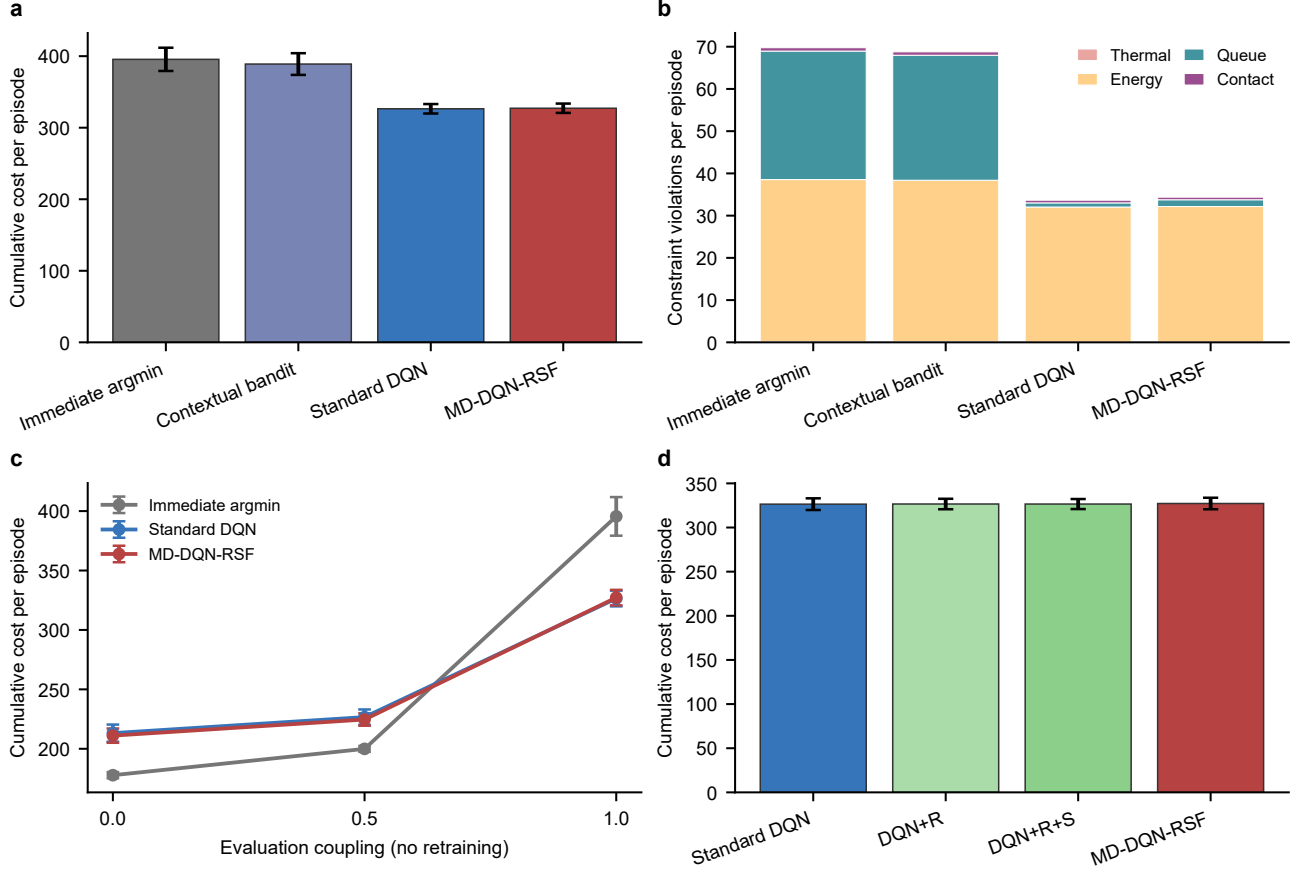


Fig. 4. Sequential scheduling performance. **a**, Nominal cumulative cost. **b**, Mean number and composition of nominal constraint violations. **c**, Evaluation under reduced action-dependent coupling without retraining; the agents were trained at $c = 1$. **d**, Cumulative RSF ablation. Error bars and shaded intervals denote SD across five independent model seeds after 20 traces were averaged within each seed; stacked bars in **b** show seed-level means.

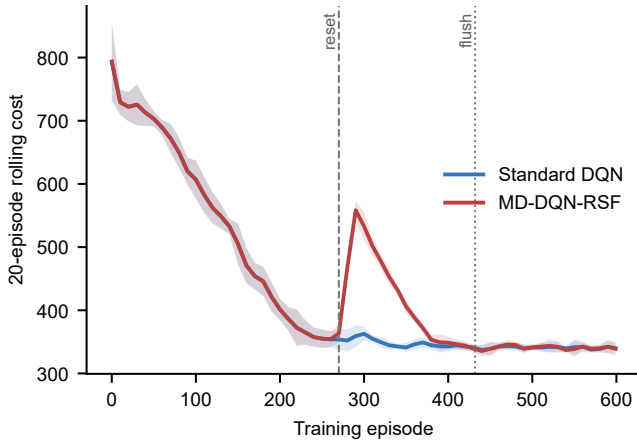


Fig. 5. Training cost for standard MD-DQN and MD-DQN-RSF. Lines show the across-seed mean of the 20-episode rolling cost and shading denotes SD ($n = 5$ independent model seeds). Vertical lines mark epsilon reset and replay-buffer flushing.

VII. DISCUSSION

The hybrid formulation assigns distinct roles to the analytical model and learned value function. The analytical

component provides an interpretable measure of the immediate resource cost of each processing mode, whereas the learned component trades a modest increase in immediate cost for fewer delayed violations. The 53.52% reduction in the penalty component, despite an 8.80% increase in immediate cost, is direct evidence of this separation. It also prevents the neural network from being presented as a substitute for evaluating three known static costs.

The need for reinforcement learning is conditional rather than universal. If tasks are independent, all relevant parameters are observed, and an action does not affect later resources, immediate-cost minimization is sufficient. This expectation was borne out at $c = 0$, and immediate argmin also remained preferable at $c = 0.5$. The learned policy became advantageous only under full coupling and remained advantageous in the four full-coupling stress tests.

The two RSF ablations refine, rather than overturn, this conclusion. The nominal schedule did not improve standard MD-DQN, while the calibration-locked schedule produced a small mean improvement whose interval crossed zero. Reset alone showed the most consistent reduction, but adding selective update and flushing did not improve the mean monotonically. These findings suggest that renewed exploration may help some training regimes, but they do not establish the complete

RSF package as uniformly superior.

The present evidence is simulation-based. The task traces are synthetic, the normalized transition coefficients have not been calibrated against spacecraft telemetry, and ground-server computation is outside the objective. The policy can therefore inherit structural error from both the analytical cost model and resource simulator. Moreover, the evaluation contains only one satellite, one ground link, and a discrete three-action space; it does not establish scalability to constellation scheduling or continuous resource allocation. Operational claims require calibration with telemetry or a high-fidelity digital twin, explicit safety constraints rather than reward penalties alone, and validation under orbital and traffic distributions not used here.

VIII. CONCLUSION

This study develops a hybrid model-driven reinforcement-learning framework for sequential task scheduling in satellite-ground edge computing. The analytical model quantifies immediate onboard, ground-based, and collaborative costs, while the value network estimates delayed resource consequences. In the fully coupled nominal simulator, MD-DQN-RSF reduced cumulative cost by 17.28% relative to immediate argmin, with a paired 95% bootstrap interval excluding zero. Most of the gain arose from fewer queue and energy violations. Immediate argmin was better when action-dependent coupling was absent or weakened. Reinforcement learning is therefore justified here by temporal resource coupling, not by the need to approximate a known static minimum. In the independent RSF confirmation, reset alone showed a small consistent benefit, whereas the full package achieved only an uncertain 0.35% mean improvement over standard MD-DQN. RSF should therefore be treated as an optional training intervention rather than the source of the sequential-learning advantage. These conclusions remain limited to the synthetic task, transition, and communication distributions evaluated in this study.

REFERENCES

- [1] Y. Pang, Y. Zhang, Y. Wang, X. Wei, and B. Chen, "Socnet: A lightweight and fine-grained object recognition network for satellite on-orbit computing," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–13, 2022.
- [2] Y. Zhang, M. Ye, G. Zhu, Y. Liu, P. Guo, and J. Yan, "Ffca-yolo for small object detection in remote sensing images," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 62, pp. 1–15, 2024.
- [3] Y. Pang, Y. Zhang, Q. Kong, Y. Wang, B. Chen, and X. Cao, "Socdet: A lightweight and accurate oriented object detection network for satellite on-orbit computing," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 61, pp. 1–15, 2023.
- [4] Y. Pang, Y. Zhang, Y. Wang, X. Wei, and B. Chen, "Exploring model compression limits and laws: A pyramid knowledge distillation framework for satellite-on-orbit object recognition," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 62, pp. 1–13, 2024.
- [5] Y. Pang, L. Yao, Y. Luo, C. Dong, Q. Kong, and B. Chen, "Repsvit: An efficient vision transformer based on spiking neural networks for object recognition in satellite on-orbit remote sensing images," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 62, pp. 1–16, 2024.
- [6] E. Casalicchio and D. Magliarisi, "Decentralized task scheduling in satellite edge computing," in *2024 9th International Conference on Fog and Mobile Edge Computing (FMEC)*, pp. 154–161, IEEE.
- [7] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," *arXiv preprint arXiv:1611.06440*, 2016.

- [8] J. Kim, J. Ahn, H.-L. Choi, and D.-H. Cho, "Task scheduling of agile satellites with transition time and stereoscopic imaging constraints," *Journal of Aerospace Information Systems*, vol. 17, no. 6, pp. 285–293, 2020.
- [9] X. Shen, Z. Lu, L. Li, Y. Chen, X. Li, J. Wang, and W. Yao, "Multi-strip stitching imaging mission planning method for sar satellites regional mapping considering on-board energy consumption," *IEEE Transactions on Geoscience and Remote Sensing*, 2024.
- [10] W. Yao, X. Shen, G. Zhang, Z. Lu, J. Wang, Y. Song, and Z. Li, "A spiking neural network based proximal policy optimization method for multi-point imaging mission scheduling of earth observation satellite," *Swarm and Evolutionary Computation*, vol. 94, p. 101867, 2025.
- [11] W. Liu, Z. Lai, Q. Wu, H. Li, Q. Zhang, Z. Li, Y. Li, and J. Liu, "In-orbit processing or not? sunlight-aware task scheduling for energy-efficient space edge computing networks," in *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*, pp. 881–890, IEEE.
- [12] Q. Jiang, H. Wang, Q. Kong, Y. Zhang, and B. Chen, "On-orbit remote sensing image processing complex task scheduling model based on heterogeneous multiprocessor," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 61, pp. 1–18, 2023.
- [13] Q. Jiang, L. Zheng, Y. Zhou, H. Liu, Q. Kong, Y. Zhang, and B. Chen, "Efficient on-orbit remote sensing imagery processing via satellite edge computing resource scheduling optimization," *IEEE Transactions on Geoscience and Remote Sensing*, 2025.
- [14] A. Abdulkarim, B. Maham, and R. C. Kizilirmak, "Multi-leo satellite networks for integrated access and backhaul: Delay analysis," in *2025 International Wireless Communications and Mobile Computing (IWCWC)*, pp. 1300–1305, IEEE, 2025.
- [15] A. Karkadakattil, "Ai-tuned hybrid thermal control of cubesats using phase change material: a matlab-based simulation study: Independent researcher, india," *Aerospace Systems*, pp. 1–10, 2025.
- [16] H. Zhang, H. Zhao, R. Liu, X. Gao, and S. Xu, "Dynamic user association and computation offloading in satellite edge computing networks via deep reinforcement learning," *IEEE Transactions on Green Communications and Networking*, vol. 8, no. 4, pp. 1888–1901, 2024.
- [17] X. Zhang, J. Liu, Z. Xiong, Y. Huang, R. Zhang, S. Mao, and Z. Han, "Cost-effective hybrid computation offloading in satellite-terrestrial integrated networks," *IEEE Internet of Things Journal*, vol. 11, no. 22, pp. 36786–36800, 2024.
- [18] R. Bellman, "Dynamic programming," *science*, vol. 153, no. 3731, pp. 34–37, 1966.
- [19] G. Tesauro, "Temporal difference learning and td-gammon," *Communications of the ACM*, vol. 38, no. 3, pp. 58–68, 1995.
- [20] C. J. Watkins and P. Dayan, "Q-learning," *Machine learning*, vol. 8, pp. 279–292, 1992.



Bocheng Yang received the B.E. degree in computer science from Wuhan University, Wuhan, China, in 2025. He is currently pursuing the Ph.D. degree with the School of Aerospace, Harbin Institute of Technology (Shenzhen), Shenzhen, China. His research interests include satellite edge computing and deep reinforcement learning.



Lujie Zheng received the B.Eng. degree in aircraft design and engineering from Dalian University of Technology, Dalian, China, in 2024. He is currently pursuing the M.Eng. degree in aeronautical and astronautical science and technology with the School of Aerospace, Harbin Institute of Technology, Shenzhen, China. His research interests include satellite edge computing and deep reinforcement learning.



Jian Liu received the Ph.D. degree from the School of Aerospace, Harbin Institute of Technology (Shenzhen), Shenzhen, China, in 2023. He is currently an Associate Researcher with the School of Aerospace. His research interests include signal processing and nonlinear filtering for urban integrated navigation systems.



Bo Chen (Member, IEEE) received the B.E. degree in aerial photogrammetry and the Ph.D. degree in photogrammetric engineering and remote sensing from PLA Information Engineering University, Zhengzhou, China, in 2002 and 2008, respectively. Since 2020, he has been a Professor with the School of Aerospace, Harbin Institute of Technology (Shenzhen), Shenzhen, China. His research interests include onboard satellite computing, geospatial big data, and spatial information engineering.